VIT®

Vellore Institute of Technology

(Deemed to be University under section 3 of UGC Act, 1956)

School of Computer Science and Engineering

BCSE309L – Cryptography and Network Security

Module ~ II – Symmetric Encryption Algorithms

RC4

Presentation by

Dr.K.Ragavan

Assistant Professor Senior Grade I

Department of IOT

School of Computer Science and Engineering

VIT University, Vellore

RC4

- RC4 (Rivest Cipher) is a stream cipher that was designed in 1984 by Ronald Rivest for RSA Data Security.
- RC4 is used in many data communication and networking protocols, including the IEEE802.11 wireless LAN standard.
- RC4 is a byte-oriented stream cipher in which a byte (8 bits) of a plaintext is exclusive-ored with a byte of key to produce a byte of a ciphertext.
- The secret key, from which the one-byte keys in the key stream are generated, can contain anywhere from 1 to 256 bytes.

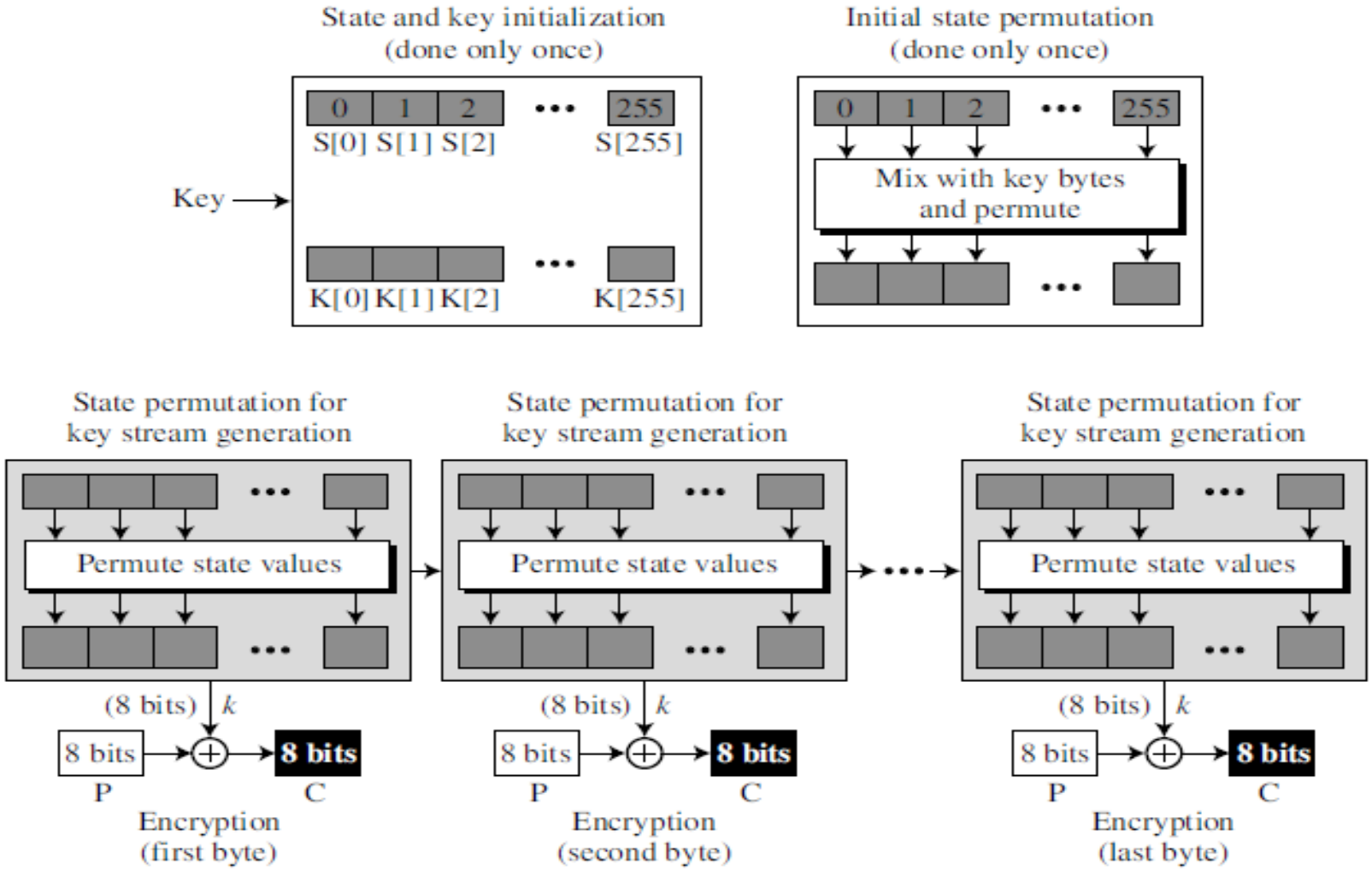
RC4 Algorithm

- It is a stream cipher with variable key size stream cipher with byte-oriented operations.
- based on the use of a random permutation.
- RC4 is used in the WiFi Protected Access (WPA) protocol that are part of the IEEE 802.11 wireless LAN standard.
- variable-length key of from 1 to 256 bytes (8 to 2048 bits) is used to initialize a 256-byte state vector S, with elements S[0],S[1], . . . ,S[255].
- At all times, S contains a permutation of all 8-bit numbers from 0 through 255. For encryption and decryption, a byte k is generated from S by selecting one of the 255 entries in a systematic fashion.
- As each value of k is generated, the entries in S are once again permuted

State

- RC4 is based on the concept of a state. At each moment, a state of 256 bytes is active, from which one of the bytes is randomly selected to serve as the key for encryption.
- The idea can be shown as an array of bytes:
    - S[0] S[1] S[2] … S[255]
- Note that the indices of the elements range between 0 and 255. The contents of each element is also a byte (8 bits) that can be interpreted as an integer between 0 to 255.

The idea of RC4 stream cipher



Step 1a) Initialization of S and T (Simple eg using 3 bits and 8 entries)

256x8bit

S = [0 1 2 3 4 5 6 7]

K = [1 2 3 6]

PLAIN = [1 2 2 2]

```
/* Initialization */
for i = 0 to 7 do
        S[i] = i;
        T[i] = K[i mod keylen];
```

Step 1b) Permutation of S with respect to T:

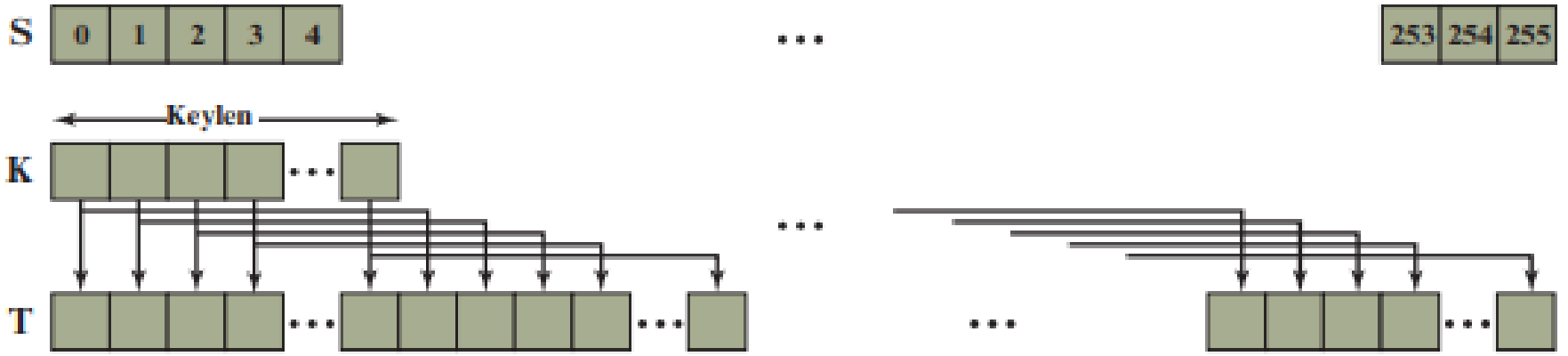
```
j=0
s = [ 0,1,2,3,4,5,6,7]
t = [1,2,3,6,1,2,3,6]
for i in range(8):
  j = (j+s[i]+t[i]) % 8
  swap(s[i],s[j])
  print("Values of i=",i,"Values of j=",j,"Modified array ",s)
```

```
Values of i= 0 Values of j= 1 Modified array  [1, 0, 2, 3, 4, 5, 6, 7]
Values of i= 1 Values of j= 3 Modified array  [1, 3, 2, 0, 4, 5, 6, 7]
Values of i= 2 Values of j= 0 Modified array  [2, 3, 1, 0, 4, 5, 6, 7]
Values of i= 3 Values of j= 6 Modified array  [2, 3, 1, 6, 4, 5, 0, 7]
Values of i= 4 Values of j= 3 Modified array  [2, 3, 1, 4, 6, 5, 0, 7]
Values of i= 5 Values of j= 2 Modified array  [2, 3, 5, 4, 6, 1, 0, 7]
Values of i= 6 Values of j= 5 Modified array  [2, 3, 5, 4, 6, 0, 1, 7]
Values of i= 7 Values of j= 2 Modified array  [2, 3, 7, 4, 6, 0, 1, 5]
```

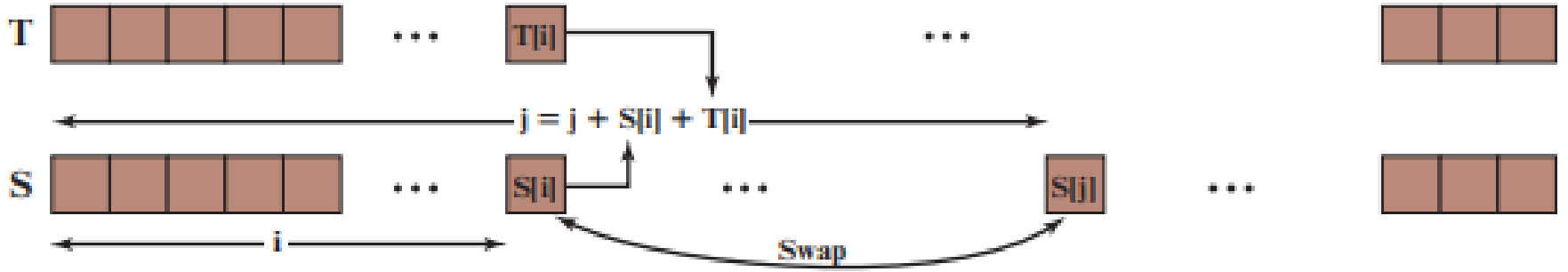
Step 2. Stream Generation

```
i, j = 0,0
plain_text = [1, 2, 2, 2]
flag =0
while (True):
  i = (i + 1) % 8
  j = (j + s[i]) % 8
  swap(s[i],s[j])
  t = (s[i] + s[j]) % 8
  k = s[t]
  cipher = k^plain_text[flag]
  print("Key value is ",k,"Plain Value is ",plain_text[flag], "Encrypted Text is ",cipher)
  flag+=1
```

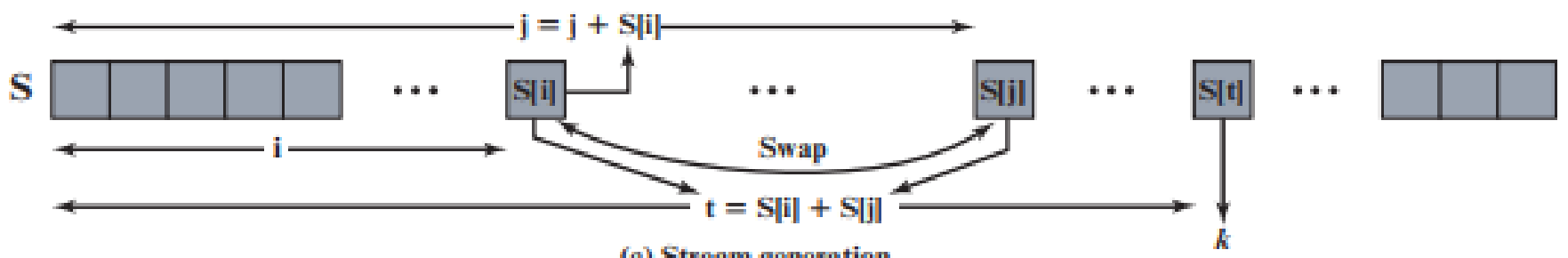
```
Key value is  5 Plain Value is  1 Encrypted Text is  4
Key value is  1 Plain Value is  2 Encrypted Text is  3
Key value is  0 Plain Value is  2 Encrypted Text is  2
Key value is  1 Plain Value is  2 Encrypted Text is  3
```



(a) Initial state of S and T



(b) Initial permutation of S



(c) Stream generation

Initialization

1. In the first step, the state is initialized to values 0, 1, …, 255.

   A key array, K[0], K[1], …, K[255] is also created.

   If the secret key has exactly 256 bytes, the bytes are copied to the K array; otherwise, the bytes are repeated until the K array is filled.

```
for (i = 0 to 255)
{
S[i] ← i
K[i] ← Key [i mod KeyLength]
}
```

Cont…

- In the second step, the initialized state goes through a permutation (swapping the elements) based on the value of the bytes in K[i].
- The key byte is used only in this step to define which elements are to be swapped. After this step, the state bytes are completely shuffled.

```
j ← 0
for (i = 0 to 255)
{
        j ← (j + S[i] + K[i]) mod 256
        swap (S[i] , S[j])
}
```

Key Stream Generation

- The keys in the key stream, the k's, are generated, one by one.
- First, the state is permuted based on the values of state elements and the values of two individual variables, i and j.
- Second, the values of two state elements in positions i and j are used to define the index of the state element that serves as k.
- The following code is repeated for each byte of the plaintext to create a new key element in the key stream.
- The variables i and j are initialized to 0 before the first iteration, but the values are copied from one iteration to the next.

$$i \leftarrow (i + 1) \bmod 256$$
$$j \leftarrow (j + S[i]) \bmod 256$$
$$\textbf{swap}\ (S[i], S[j])$$
$$k \leftarrow S[(S[i] + S[j]) \bmod 256]$$

Encryption or Decryption

- After k has been created, the plaintext byte is encrypted with k to create the ciphertext byte.
- Decryption is the reverse process.

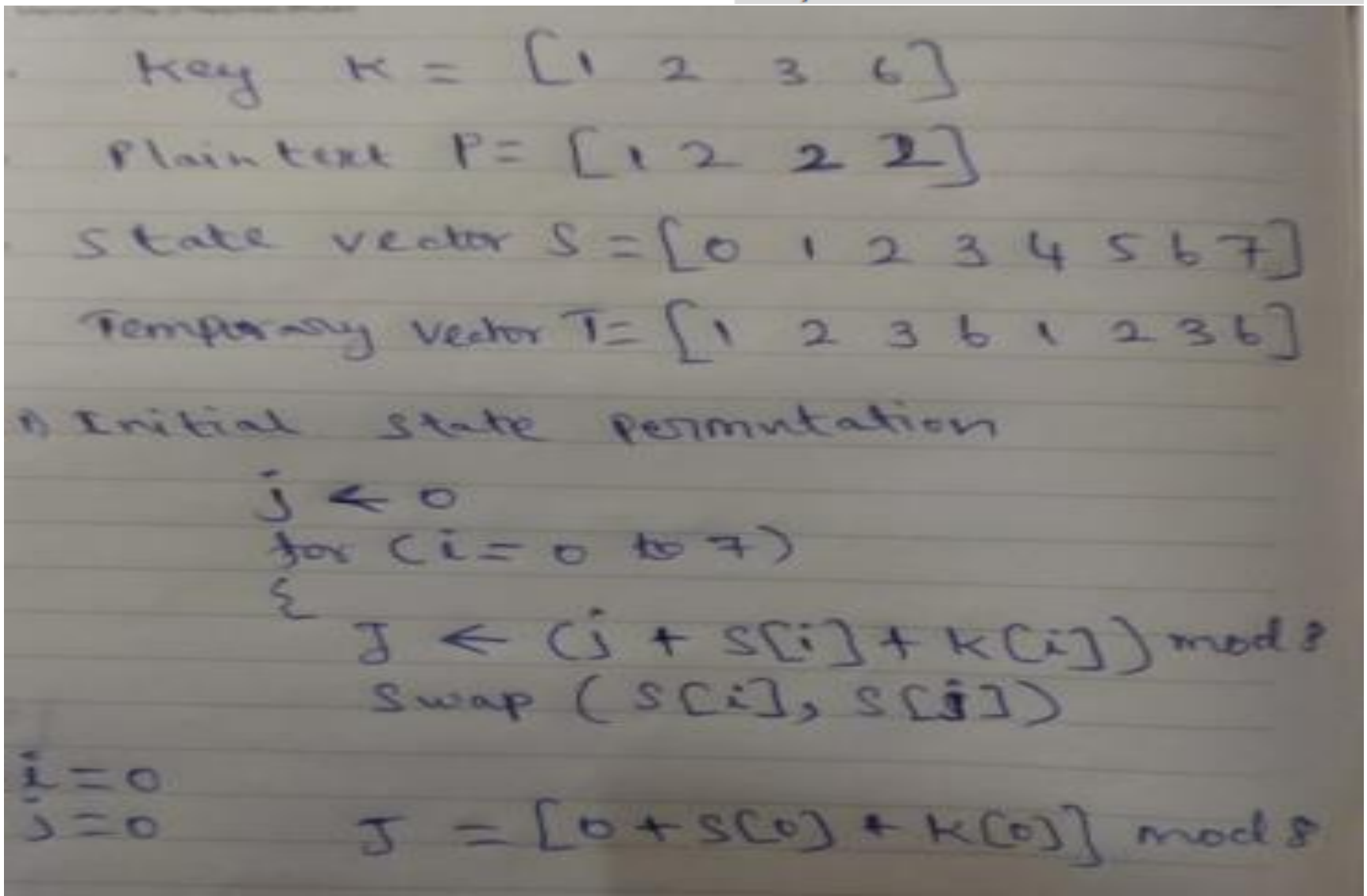
```
// Key is ready, encrypt
input P
C ← P ⊕ k
output C
```

Security Issues

- It is believed that the cipher is secure if the key size is at least 128 bits (16 bytes).
- There are some reported attacks for smaller key sizes (less than 5 bytes), but the protocols that use RC4 today all use key sizes that make RC4 secure.
- However, like many other ciphers, it is recommended the different keys be used for different sessions. This prevents Eve from using differential cryptanalysis on the cipher.

Example Problem

```
j ← 0
for (i = 0 to 255)
{
        j ← (j + S[i] + K[i]) mod 256
        swap (S[i] , S[j])
}
```



```
j ← 0
for (i = 0 to 255)
{
        j ← (j + S[i] + K[i]) mod 256
        swap (S[i] , S[j])
}
```

S = [0 1 2 3 4 5 6 7]
S[0] S[1] S[2] S[3] S[4] S[5] S[6] S[7]

Here K indicates Tamporary vector T

So, T = [1 2 3 6 1 2 3 6]
K[0] K[1] K[2] K[3] K[4] K[5] K[6] K[7]

J = [0 + 0 + 1] mod 8
j = 1 mod 8 = 1

i = 0 j = 1

Swap (S[0], S[1])

updated State Vector

S = [1 0 2 3 4 5 6 7]
S(0) S(1) S(2) S(3) S(4) S(5) S(6) S(7)

```
j ← 0
for (i = 0 to 255)
{
        j ← (j + S[i] + K[i]) mod 256
        swap (S[i] , S[j])
}
```

$i = 1, j = 1$

$$j = [j + S[i] + K[i]] \bmod 8$$

$$j = [1 + S(1) + K(1)] \bmod 8$$

$$j = (1 + 0 + 2) \bmod 8$$

$$j = 3 \bmod 8$$

$i = 1$ $\quad$ $j = 3$

swap (S[i], S[j])

Swap (S[1], S[3])

$S = [1 \quad 3 \quad 2 \quad 0 \quad 4 \quad 5 \quad 6 \quad 7]$

$S[0] \; S[1] \; S[2] \; S[3] \; S[4] \; S[5] \; S[6] \; S[7]$

Initial state permutation

- Repeat the steps until i value reaches the 7^{th} iteration
- After updating the state vector, every time, i value will be incremented by 1.
- But, j value should not be incremented, every iteration, the last j value will be used.

S = [1 0 2 3 4 5 6 7]
T = [1 2 3 6 1 2 3 6]

$i, j = (j + S[i] + T[i]) \bmod 8$	
i = 0, j = 1	S = [1 0 2 3 4 5 6 7]
i = 1, j = 3	S = [1 3 2 0 4 5 6 7];
i = 2, j = 0	S = [2 3 1 0 4 5 6 7];
i = 3, j = 6;	S = [2 3 1 6 4 5 0 7];
i = 4: j = 3	S = [2 3 1 4 6 5 0 7];
i = 5, j = 2	S = [2 3 5 4 6 1 0 7];
i = 6, j = 5;	S = [2 3 5 4 6 0 1 7];
i = 7, j = 2;	S = [2 3 7 4 6 0 1 5];

Hence, our initial permutation of S gives: S = [2 3 7 4 6 0 1 5];

$i \leftarrow (i + 1) \bmod 256$

$j \leftarrow (j + S[i]) \bmod 256$

swap ($S[i]$, $S[j]$)

$k \leftarrow S[(S[i] + S[j]) \bmod 256]$

2) Key stream Generation:

updated initial permutation

S = [2 3 7 4 6 0 1 5]

S[0] S[1] S[2] S[3] S[4] S[5] S[6] S[7]

initially i = 0, j = 0

i = (i + 1) mod 8

= (0 + 1) mod 8

i = 1

j = (j + S[i]) mod 8

= (0 + S[1]) mod 8

= (0 + 3) mod 8

j = 3

26

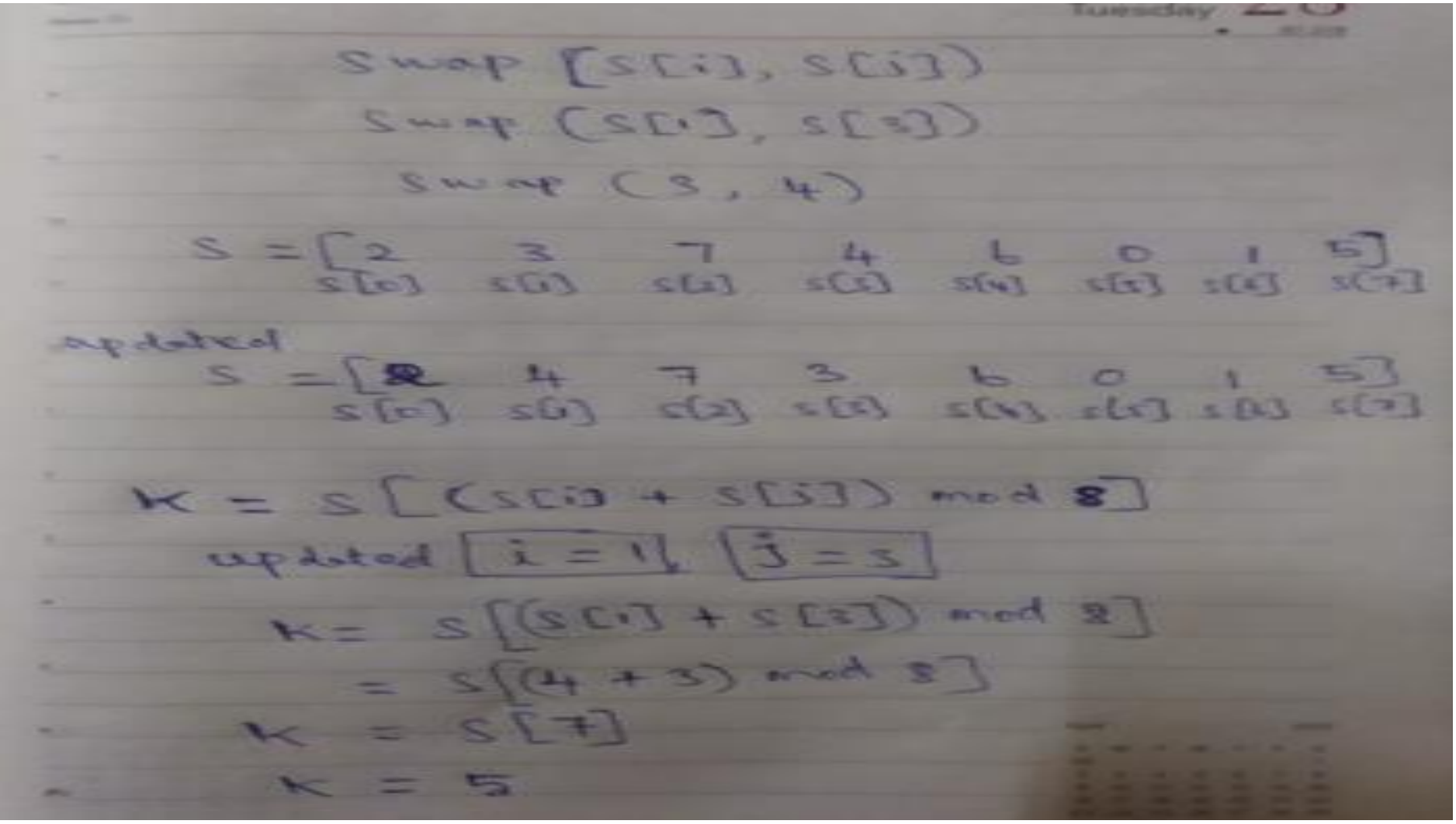
Sunday

$i \leftarrow (i + 1) \bmod 256$

$j \leftarrow (j + S[i]) \bmod 256$

swap (S [i] , S[j])

$k \leftarrow S\,[(S[i] + S[j]) \bmod 256]$



Key stream generation

- But here, Every iteration, updated i and j values will be used.
- i and j values will not be incremented

3) Generate Ciphertext

$$C = P \oplus K$$

So, generate 3 bits

$P = [1\ 2\ 2\ 2]$

$C = 1 \oplus 5$

$= 001 \oplus$

101

$C = 100$

$\boxed{C = 4}$

iteration=1, k=5	5 XOR 1 = 101 XOR 001 = 100 = 4
iteration=2, k=1	1 XOR 2 = 001 XOR 010 = 011 = 3
iteration=3, k=0	0 XOR 2 = 000 XOR 010 = 010 = 2
iteration=3, k=1	1 XOR 2 = 001 XOR 010 = 011 = 3

P = [1 2 2 2]
K = [5 1 0 1]
C = [4 3 2 3]